SightLine

It reads your documents aloud — on your phone, with no network.

Team STEMB · Imperial Hackathon Hong Kong & Macau 2026 — Grand Final

Live: adeviwon.github.io/SightLine

THE MOMENT WE BUILT THIS FOR

The prescription is on the table, and she cannot read it.

She is 80. The dose is two lines down, in type she has not been able to see for years.

She is alone in the flat, so it gets guessed — or it waits for a neighbour who is not coming today.

No statistic on this slide. It is the reason the next one matters.

THE PROBLEM AT SCALE

199,600 people

live with visual impairment in Hong Kong — **2.7% of the population**. 113,900 of them (57.1%) live with multiple disabilities.

Every cloud reader on the market asks them to upload that prescription first. For a medication label, offline is not a feature — it is the only answer that holds.

[source: docs/13_ROUND1_SUBMISSION_KIT.md — HKSAR Census and Statistics Department, Special Topics Report No. 63, *Persons with disabilities and chronic diseases* (December 2021, 2020 survey round), Appendix 4, via the Hong Kong Blind Union]

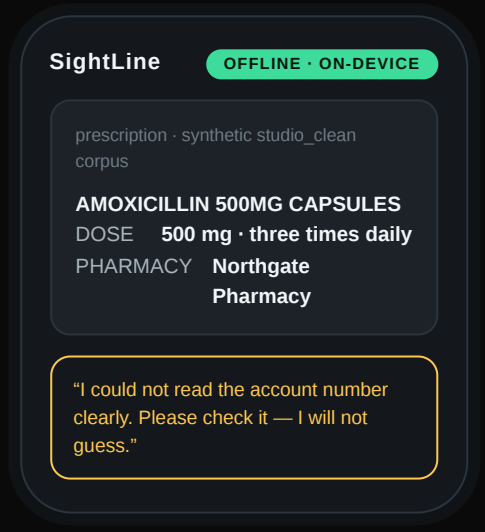
THE SOLUTION

Point the phone at the document. It reads it aloud, speaks the fields that matter, and refuses what it cannot read.

Our own recogniser: **942,166 parameters**, 37 symbols (0-9, A-Z, space). No OCR library, no cloud, no account.

26.7 MB of ONNX on the device. Airplane mode: nothing leaves the phone.

Chinese is not in the alphabet yet. We say that publicly, and it is roadmap item 1.



[source: artifacts/ocr_training.json (params 942166, charset 37 symbols) · HANDOFF.md §payload (26.7 MB installable) · app/index.html (design tokens and UI) · app/tools/render_samples.py (the sample document is synthetic, not a real patient’s)]

LIVE DEMO

Airplane mode on. Watch it read.

1. Open **adeviwon.github.io/SightLine** on this phone — installed, then reloaded with the network off.
2. **Airplane mode ON.** Wi-Fi off, data off. Nothing is uploaded; there is nowhere for it to go.
3. Scan the prescription. Hear it read the medication and the dose aloud.
4. Scan the second document. Watch it **refuse** one field and say why, instead of guessing.

> Fallback if the phone or the room fails: the same pipeline runs from the repo (**bash run.sh serve**), and the 12-document bench replays from **artifacts/pipeline_raw_deskew.json**.

[source: docs/05_DEMO_RUNBOOK.md]

HOW IT WORKS



Hard part 1 — inference in a browser. ONNX Runtime Web, our own graph, 26.7 MB shipped to the phone. No OCR library, no server, no fallback engine.

Hard part 2 — the refusal threshold. A field below confidence is omitted, never guessed: 15 of the 24 ground-truth fields on our bench are missing rather than wrong.

[source: app/js/pipeline.js · app/js/field_guard.js · app/js/recognizer.js · artifacts/ocr_training.json · artifacts/pipeline_raw_deskew.json]

MEASURED RESULTS, HONESTLY

Tesseract wins on clean flat scans — we win 5.5× on hand-held captures.

SAME 12 DOCUMENTS · SAME CROPS · SAME GROUND TRUTH (188 LINES)	LINE EXACT	WORD ACC	CER	FIELDS
Tesseract 4.1.1 — whole page, `--psm 3` (its fair integration)	72.9%	80.1%	0.134	79.2%
Tesseract 4.1.1 — native-resolution line crops (its fair input)	64.4%	75.3%	0.164	70.8%
Tesseract — the *identical* 32 px crops we read (unfair control, labelled)	17.6%	49.8%	0.357	29.2%
ours — CRNN on 32 px crops (what ships)	**11.7%**	**61.2%**	**0.197**	**83.3%**

> **Plainly:** Tesseract is stronger on clean, flat scans; we are stronger on real hand-held captures, we never invent a punctuation mark or a digit, and we degrade instead of hallucinating.
> **Where the split lands:** on **handheld_light** — the one profile that models a real hand-held capture — ours reads **68.4% word accuracy / 83.3% of fields** against Tesseract's **12.4–23.6% / 0–16.7%**, a 5.5× gap.
> **Fairness correction, stated not hidden:** our 37-symbol alphabet (0-9, A-Z, space) cannot emit punctuation at all, and 120 of the 188 scored rows contain it — that alone is the 11.7% → 41.0% line-exact gap; on the identical-pixel control, punctuation-blind, ours takes **48 lines** Tesseract misses, and loses 16. **Safety:** on the clean page Tesseract read the dosage **500mg as 300mg** — the exact field this product exists to protect.
> **Traceable record:** `artifacts/pipeline_raw_deskew.json` — 12 records, one per document, each `error: null` with its read text (the only verification count we quote; the CPU-starved browser bench is kept labelled in `artifacts/webkit_bench.log`).

[source: docs/24_ENGINE_HEAD_TO_HEAD.md · artifacts/engine_head_to_head.json — 12 documents, 188 ground-truth lines, Tesseract 4.1.1 · shipped end-to-end 45.9% word acc / CER 0.303, from 3.6% (`artifacts/pipeline_raw_deskew.json`, `pipeline_raw.json`) · hard case 20 real hand-photographed pages, 1.8% line-exact (`docs/14_HACKATHON_BENCHMARK.md`)]

WHAT WE GOT WRONG — AND FIXED

Four train/serve contract defects. We found them in our own code, measured them, and gated them.

1 · the graph was baked at 256 px; training crops are clamped to 320. The app fed the model the first 256 px of 98.8% of lines. 3.6% → 22.4% word accuracy; 200/200 crops changed their read.

2 · padding is not inert in a bidirectional CRNN. The backward pass resumes at the right edge, inside the pad, so zeros moved the hidden states across valid frames. 30.5% vs 19.0% crop-exact; CER 0.198 vs 0.308.

3 · the candidate list replaced the preprocessed image. Only CLAHE arms were offered, and raw deskewed grey measured best. 26.9% vs 23.9% word accuracy — the best input was unreachable.

4 · the deskew sign was inverted. We rotated by the estimate and doubled the tilt instead of removing it. 4.5° applied read back as 6.5–9.0°; it zeroed the two marginal off-axis pages.

> Each defect now has a gate that fails if it returns: **test_graph_width_contract.py** · **test_padding_invariant.py** · **crop_contract.js** · **deskew_verify.js**.

[source: docs/18_CRNN_ACCURACY_STRATEGIES.md §0.1 · docs/14_HACKATHON_BENCHMARK.md (256 vs 320) · app/js/pipeline.js (candidate list, deskew sign)]

IMPACT AND POLICY

46.4% of older Hong Kong people

take five or more medicines at once — the highest of Hong Kong, Taiwan, South Korea, the UK and Australia (2016), and rising 2.7% a year. Polypharmacy plus sight loss is our use case.

¶75 · 2025 Policy Address — AI application: "…render carer support through an AI chatbot to give carers more convenient access to information", and generative AI in clinical workflows. A carer hearing their own document read out, on the phone they already own.

¶227 · 2025 — healthcare reform for an ageing society (¶226 opens the section): "enhancing ophthalmic services … a new high-capacity ambulatory cataract surgery centre". The only directly vision-facing commitment in either edition.

¶266 · 2025 — Working Group on Ageing Society Strategies covering "…elderly care, healthcare, housing, culture and leisure, and gerontechnology"; and "…by 2046, people aged 65 and above will account for **36%** of our total population".

2026 is the funded follow-through, not the anchor: ¶113 allocates **$300 million** to the "AI for Welfare Lab"; ¶398–400 earmark **$100 million** for the Gerontechnology Promotion Scheme.

[source: docs/19_POLICY_ANCHOR.md — the **2025** Policy Address paras 75 / 227 / 266 quoted verbatim from `policy-full_en.pdf`, sha256 c4ecaf20…18f3ec (the edition the Open Challenge rule names, official FAQ read 2026-10-21); 2026 follow-through from the 2026 PDF, sha256 f24973cd…49246b · docs/12_HACKATHON_SUBMISSION.md — 46.4%, Lee H. et al., *Age and Ageing* 2023, doi:10.1093/ageing/afad014]

It reads it —
or it says it cannot.

Live: adeviwon.github.io/SightLine

Repo: github.com/adeviwon/SightLine

Team STEMB — T on-device ML · E restoration · S accessibility research · M medication safety · B deployment